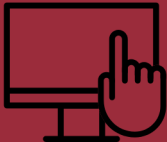




AULA 3

Técnica aplicada

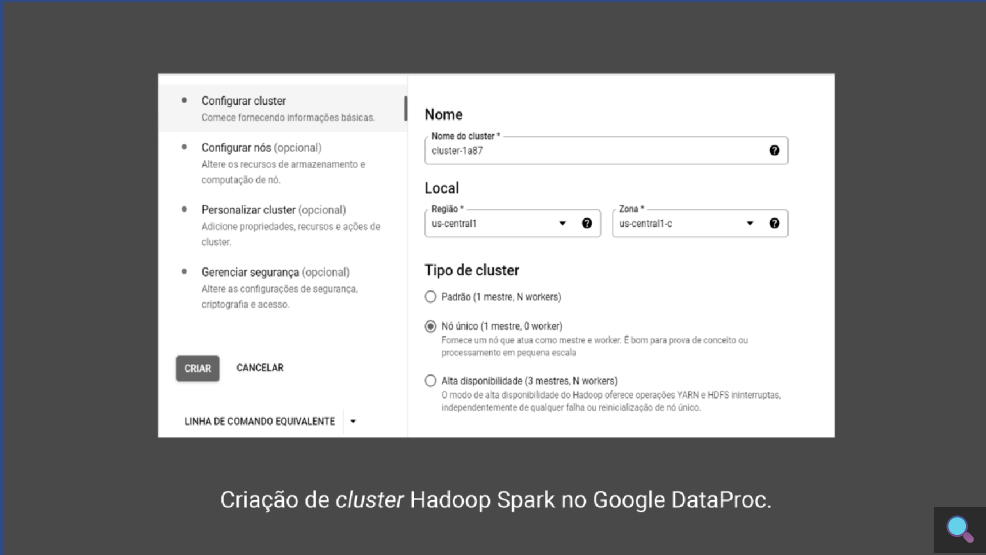


Mão na massa

Nesta seção, utilizaremos as técnicas e ferramentas mostradas na aula para implementar uma aplicação de um DL em um *cluster* Hadoop Spark. Todos os provedores de nuvem oferecem plataformas de dados baseadas em Hadoop Spark, como, por exemplo, AWS Elastic MapReduce (AWS EMR), Azure HDInsight, Oracle Big Data Service e Google DataProc. Vamos utilizar este último para criar um *cluster* Hadoop, ingerir dados para o HDFS e utilizar MapReduce e Hive para executar *jobs* e consultas sobre os dados ingeridos, mas você pode usar qualquer outro provedor de nuvem de sua escolha.

[Clique aqui](#) para acessar!

Para instalar um *cluster* Hadoop Spark na Google, acesse o seu Gmail e, em seguida, acesse <https://cloud.google.com>. Nessa nuvem, assim como em outras, você poderá utilizar créditos para implantar e utilizar os serviços, dentro de períodos pré-estabelecidos. No *menu* à esquerda da *console*, procure por DataProc. Na página de criação do *cluster*, dê um nome, selecione a região e escolha qual tipo de *cluster* será criado. Um *cluster* padrão terá um *master node* e *n workers*. Um *cluster* de nó único pode ser usado para a realização de provas de conceito e experimentações. E o tipo *high availability* permite o uso de três *master nodes* com *n workers*, para cargas com alto volume e alta demanda de processamento. Vamos utilizar um *cluster* de nó único.



Na mesma página, na parte inferior, selecione os componentes que serão utilizados: Hive e Jupyter *notebooks*.



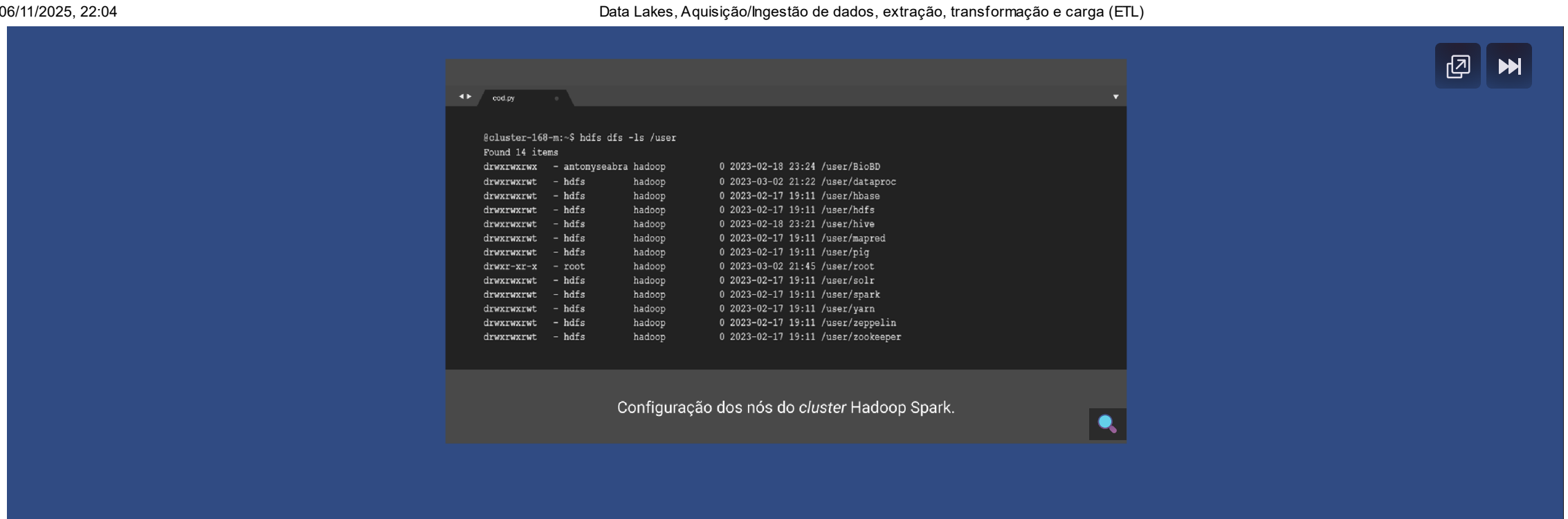
Selecione a função Configurar nós para informar a configuração de *hardware* dos nós do *cluster*. No nosso caso, como temos apenas um nó do *cluster*, está disponível a configuração do nó administrador. Utilize a configuração padrão, alterando apenas o comando do disco principal para 50 Gbytes.



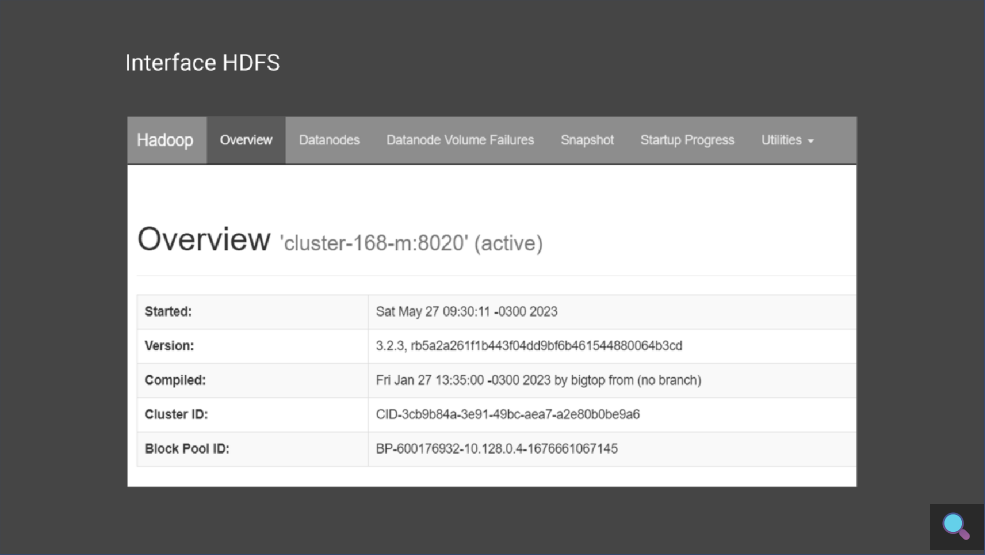
Clique em Criar e aguarde o provisionamento do *cluster*. Uma vez que o painel mostre o *cluster* Em execução, selecione o *cluster* para acessar seu painel de monitoramento. Aí são mostrados dados de uso de CPU, memória, rede e HDFS do *cluster*, que serão coletados durante o processamento dos *jobs* e consultas.

Usando HDFS

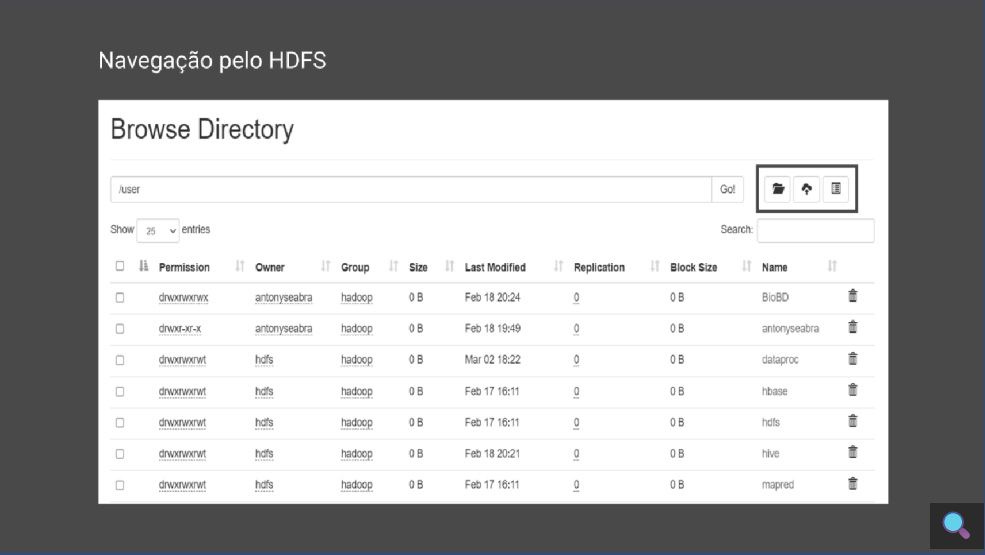
Em Instâncias da VM, abra um terminal SSH para o *master node*. Uma janela será aberta, e é possível executar comandos HDFS para listar o conteúdo de um diretório, listar o conteúdo de um arquivo e realizar operações tradicionais, como mover e copiar arquivos. Veja, a seguir, como listar o conteúdo do diretório padrão /user.



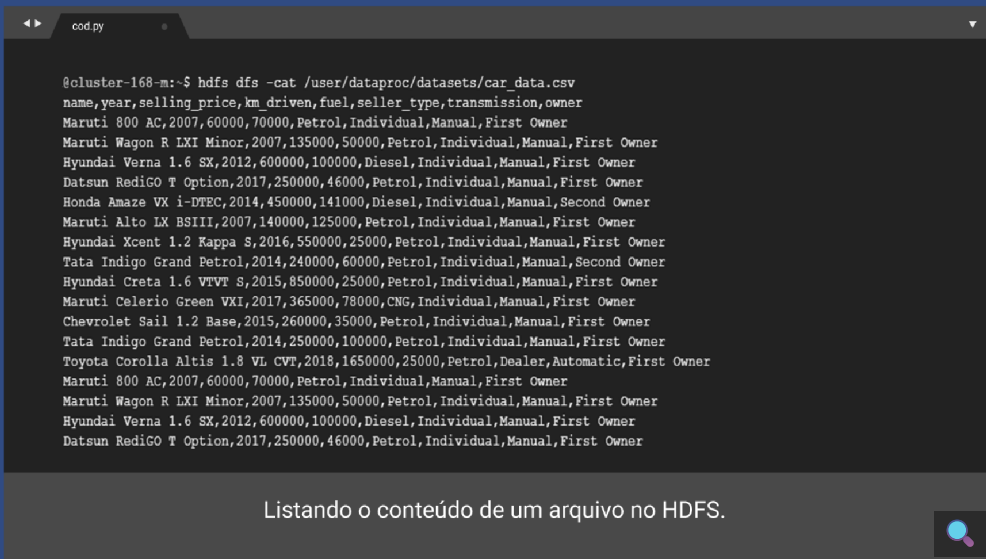
Para copiar arquivos para o HDFS, pode-se utilizar a API REST WebHDFS ou a interface gráfica disponível no sistema. É possível utilizar também os *buckets* da nuvem Google e o comando *cp*. Se estiver utilizando uma estação local com sistema operacional Linux, você pode também utilizar *copyFromLocal*. Para acessar a interface gráfica do HDFS, em Interfaces da Web (ao lado de Instâncias da VM), clique em HDFS Namenode.



No *menu* Utilities, selecione Browse the file system, navegue pelo sistema de arquivos e localize a pasta destino para onde deseja copiar o arquivo e clique no botão de *upload* no canto superior direito. Escolha o arquivo local e aguarde o término da cópia.



Uma vez que o arquivo esteja copiado, utilize o comando *cat*, como mostrado a seguir.

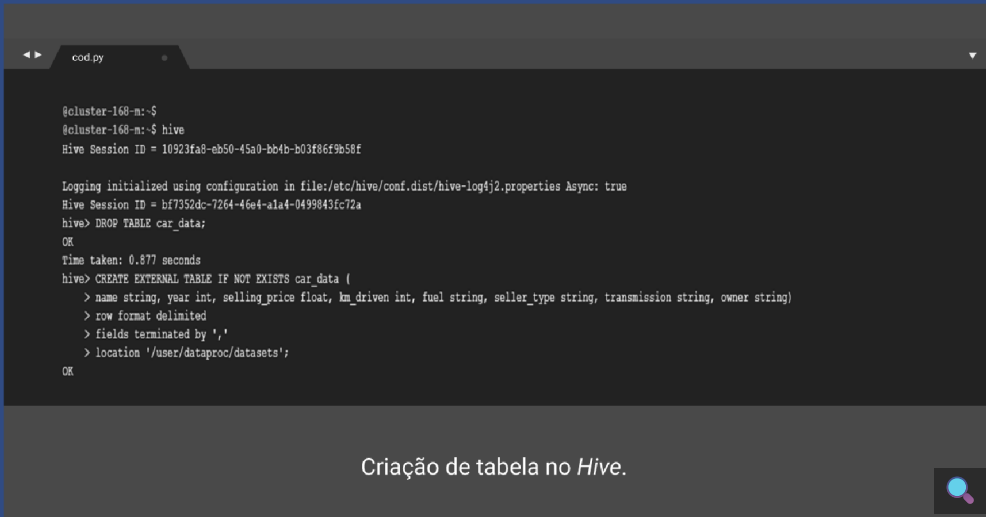


Dados semiestruturados e não estruturados estão presentes em muitas fontes de dados disponíveis para as organizações. Bons exemplos de fontes de dados no Brasil incluem o Portal de Dados Abertos do Governo Federal (<https://dados.gov.br/>), que oferece dados de várias áreas em formato CSV, JSON, HTML, entre outros; o Portal de Dados do IBGE, que disponibiliza dados de geociências em imagens e do Censo Brasileiro em formato *shape file*; portais de organizações não governamentais e universidades, como o [MapBiomias](#), que disponibiliza dados sobre meio ambiente e desmatamento, um tema cada vez mais relevante em nossa sociedade; ou, ainda, o portal [Base dos Dados](#), que reúne um conjunto de coleções de dados e serviços de acesso para diversos domínios.

Como comentamos anteriormente, você pode copiar os dados para o HDFS em sua forma bruta, ou *raw*, e utilizar o componente do ecossistema Hadoop mais adequado ao tipo de processamento a ser executado. Por exemplo, o arquivo *car_data.csv* mostrado na figura anterior pode ser utilizado para a criação de uma tabela no Hive e para que seja possível executar consultas SQL sobre ele.

Usando Hive

Hive é uma *metastore* criada sobre o HDFS que oferece uma linguagem padrão SQL, denominada HiveQL. Suas tabelas podem ser internas ou externas. Os dados de uma tabela interna são armazenados na *metastore*, e os dados de uma tabela externa são armazenados no próprio HDFS. Por padrão, a *metastore* do Hive é um SGBD MySQL. Na imagem a seguir, criamos uma tabela externa *car_data*, indicando os tipos das colunas e apontando para o arquivo residente no HDFS.



Após a criação da tabela, podemos executar comandos SQL, como a contagem de linhas da figura a seguir.

cod.py

```
hive> SELECT COUNT(*) FROM car_data;
Query ID = netwmyneabra_20230527131541_734e074e-17df-44a3-8e4a-e74b5e606778
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Running on YARN cluster with App id application_1685190636487_0002)
```

VERTICES	WEEK	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	7	7	0	0	0	0
Reducer 2	container	SUCCEEDED	1	1	0	0	0	0

```
VERTICES: 02/02 [----->>] 100% ELAPSED TIME: 11.83 s
=====
OK
5029913
Time taken: 16.683 seconds, Fetched: 1 row(s)
```

Execução de comando SQL no Hive.

Em seguida, executamos um SQL para recuperar as linhas da tabela que satisfazem ao predicado dado, composto pelas colunas *name* e *year*.

cod.py

```
hive> SELECT * FROM car_data WHERE name LIKE 'Hyundai*' and year=2017;
Query ID = netwmyneabra_202305271430_57c7b761-8ba8-41a8-811f-8a4f5a3b1119
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Running on EMR cluster with App id application_14704127120_0014)
```

VERTICES	WEEK	STATUS	TOTAL	COMPLETED	RUNNING	PENDING	FAILED	KILLED
Map 1	container	SUCCEEDED	7	7	0	0	0	0

```
VERTICES: 01/01 [----->>] 100% ELAPSED TIME: 21.01 s
=====
OK
```

```
Hyundai Accent SE 4pt Hatchback 2017 1190000.0 5000 Hatchback Dealer Manual First Owner
Hyundai Accent I10 1.2 Regio Asia 2017 580000.0 15100 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.4 CRDi 2017 750000.0 20000 Hatchback Dealer Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 300000.0 31000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 650000.0 22000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 33000 110000.0 15000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 700000.0 18000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 140000.0 20000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 580000.0 32100 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 300000.0 11140 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 650000.0 30000 Hatchback Individual Manual Second Owner
Hyundai Accent I10 1.6 Regio Asia 2017 220000.0 33000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 300000.0 21000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 400000.0 24000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 CRDi 2017 700000.0 31000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 600000.0 32100 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 200000.0 33000 Hatchback Individual Manual First Owner
Hyundai Accent I10 1.6 Regio Asia 2017 550000.0 17000 Hatchback Individual Manual First Owner
```

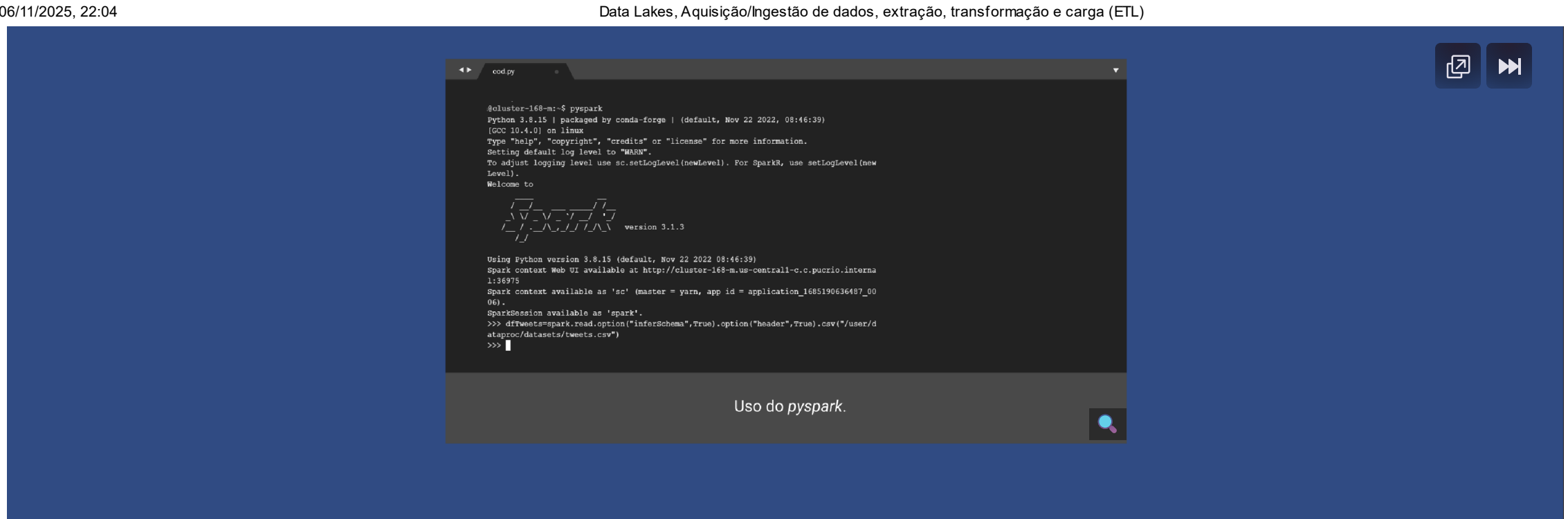
Execução de comando SQL no Hive.

Podemos observar, pelos *outputs* das execuções Hive, que o primeiro SELECT foi executado por tarefas Map e Reduce, enquanto o segundo SELECT tem apenas tarefa Map. Isso ocorre porque o primeiro SELECT apresenta uma agregação, no caso a contagem de linhas, executada por uma tarefa Reduce, enquanto o segundo SELECT apresenta apenas a recuperação das linhas com aplicação de filtro, executada por uma tarefa Map.

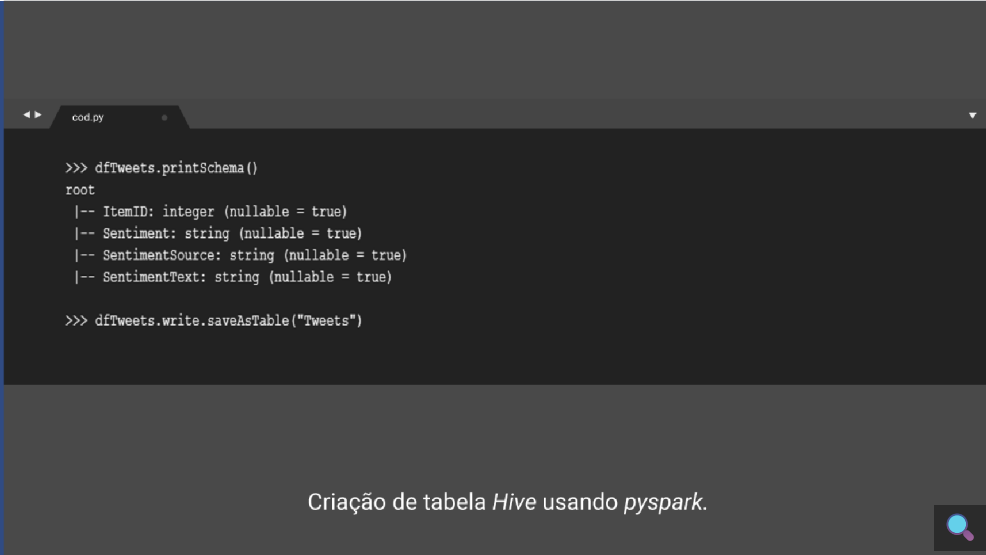
Combinando Spark, Hive e MapReduce para analisar dados do Twitter

Consultas escritas em HiveQL são submetidas ao Hive Server. O Hive, então, transforma essas consultas em tarefas MapReduce, que processa as tarefas sobre os arquivos HDFS e retorna os resultados para o Hive. Na configuração de um DL, existem alternativas ao MapReduce. Uma delas é o Tez, que é uma estrutura para a criação de um gráfico acíclico dirigido (DAG) complexo de tarefas para o processamento de dados distribuídos. Outra, como visto anteriormente, é o Spark, que utiliza estruturas em memória (RDDs ou *dataframes*) para executar as tarefas MapReduce.

No exemplo que se segue, vamos utilizar a integração no ecossistema Hadoop para acessar o Hive a partir do Sparke para trabalhar com dados coletados do Twitter. Após a cópia do arquivo *tweets.csv* para o HDFS, execute o pyspark para abrir uma sessão Spark no *cluster* Haddop e um shell Python. Em seguida, crie um *dataframe* a partir do arquivo *tweets.csv*, com inferência de esquema e utilizando a primeira linha do arquivo CSV como identificador das colunas.



Veja, na imagem a seguir, o esquema inferido. A seguir, utilizamos `saveAsTable` para criar uma tabela a partir do `dataframe`. Essa `table` será criada como uma tabela interna do Hive.



Usando SPARK-SQL, podemos realizar consultas sobre as tabelas do Hive. No exemplo a seguir, recuperamos a quantidade de `tweets` positivos e a quantidade de `tweets` negativos (coluna `Sentiment`).



Agora, queremos responder a pergunta: **Quais são as 10 palavras mais utilizadas em tweets com sentimento positivo?**





Converter o *dataframe* em um RDD permite usá-lo como entrada para um trabalho MapReduce. Adicionalmente, definimos uma lista de *stopwords*, que são termos que não devem ser considerados em uma contagem, por serem muito frequentes em um idioma (é o caso de artigos, preposições, conjunções, por exemplo).

cod.py

```
>>> stopWords = ['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', 'her', 'hers', 'herself', 'it', 'its', 'itself', 'they', 'them', 'their', 'theirs', 'them selves', 'what', 'which', 'who', 'whom', 'this', 'that', 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'in to', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'd own', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 's o', 'than', 'too', 'very', 'can', 'will', 'just', 'don', 't', 's', 'should', 'now']
>>>
```

Stop words a serem desconsideradas na contagem de palavras.

Em seguida, podemos executar um *job* que separa as palavras, desconsiderando as *stopwords* listadas, criando uma estrutura chave-valor que passa pelas fases Map e Reduce para obter uma contagem de cada palavra.

cod.py

```
>>> contadorPalavras = rddPos.map(lambda x:x.SentimentText.replace(' ','').replace('-', '').replace('_', '').lower() \
... .flatMap(lambda x: x.split()) \
... .filter(lambda x: x not in stopWords) \
... .map(lambda x: (x, 1)) \
... .reduceByKey(lambda x,y:x+y) \
... .map(lambda x:(x[1],x[0])) \
... .sortByKey(False)
>>>
>>>
>>> contadorPalavras.take(10)
[(1839, 'good'), (1827, 'i'm'), (1650, 'love'), (1419, 'like'), (1250, 'lol'), (1231, 'thanks'), (1195, 'get'), (1186, 'u'), (1113, 'it's'), (1037, 'know')]
>>> █
```

Job MapReduce para contagem de palavras mais frequentes.

Note que inicialmente a função *split* separa os termos na variável x, descartando aqueles que estão na lista de *stopwords*. Para cada palavra, a estrutura chave-valor é criada pela função *lambda*, inicialmente como (x,1). O *reducer* soma os valores das estruturas que têm a mesma chave, resultando em outra estrutura com a contagem total de cada termo. A função *take(10)* retorna os 10 termos mais citados nos tweets positivos, iniciando com *good* e terminando com *know*.

Usando Google Colab para instanciar sessão Spark

Nesta seção, vamos implantar a mesma solução usando Google Colab para instanciar uma sessão Spark, importar os dados de tweets e realizar a contagem. Inicialmente, devemos instalar o pyspark, conforme abaixo:

cod.py

```
[1] !pip install pyspark --quiet

[2] from pyspark import SparkContext, SparkConf
from pyspark.sql import SparkSession

spark = SparkSession.builder.master("local[*]").getOrCreate()

conf = SparkConf().set('spark.ui.port', '4050').setAppName("twitter").setMaster("local[*]")
sc = SparkContext.getOrCreate(conf=conf)
```

Abrindo sessão Spark no Google Colab.



A segunda célula cria a sessão e o contexto Spark, que usaremos adiante. Em seguida, montamos o Google Drive, para onde devemos copiar o arquivo *tweets.csv*. Depois, fazemos exatamente como antes: criamos um *dataframe* com inferência de esquema e utilizando a primeira linha com os nomes das colunas.

```
[3] # Load the Drive helper and mount
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[4] url = "/content/drive/My Drive/Colab Notebooks/DL/tweets.csv"
df = spark.read.option("inferSchema",True).option("header",True).csv(url)
df.printSchema()

root
 |-- ItemID: integer (nullable = true)
 |-- Sentiment: string (nullable = true)
 |-- SentimentSource: string (nullable = true)
 |-- SentimentText: string (nullable = true)
```

Criação de *dataframe* Spark.

Na célula 5, salvamos a tabela *tweets* e, na célula 6, executamos o **SELECT** contendo o número de ocorrências de cada *Sentiment*.

```
[5] df.write.saveAsTable("tweets")

[6] spark.sql("SELECT Sentiment, COUNT(*) FROM tweets GROUP BY Sentiment").show()

+-----+-----+
|Sentiment|count(1)|
+-----+-----+
|pos|26921|
|0|2|
|neg|23079|
+-----+-----+
```

Consulta SQL na sessão Spark.

Aqui utilizamos o contexto Spark criado anteriormente para executar o código que transforma o *dataframe* em RDD, habilitando o uso do MapReduce.

```
[7] from pyspark.sql import HiveContext
hivecontext = HiveContext(sc)
hiveQuery = "SELECT SentimentText FROM tweets WHERE Sentiment='pos';"
dfPos = df = hivecontext.sql(hiveQuery)
rddPos = dfPos.rdd
```

Abrindo sessão Spark no Google Colab.

E, por fim, setamos a variável *stopwords* e executamos o código MapReduce para realizar a contagem de palavras sobre o RDD. A célula 10 mostra essa contagem para as 10 primeiras palavras.

```
[8] stopWords = ['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his',

[9] contadorPalavras = rddPos.map(lambda x:x.SentimentText.replace(',','').replace('.', ' ').replace('-', '').lower()) \
.flatMap(lambda x: x.split()) \
.filter(lambda x: x not in stopWords) \
.map(lambda x: (x, 1)) \
.reduceByKey(lambda x,y:x+y) \
.map(lambda x:(x[1],x[0])) \
.sortByKey(False)

contadorPalavras.take(10)

[(1839, 'good'),
(1827, 'i'),
(1650, 'love'),
(1419, 'like'),
(1250, 'lol'),
(1231, 'thanks'),
(1195, 'get'),
(1186, 'u'),
(1113, 'it's'),
(1037, 'know')]
```

Contagem de palavras na sessão Spark.



Fique Ligado

Todo provedor de nuvem disponibiliza uma quantidade limitada de créditos gratuitos para uso e é fundamental que todos utilizem esses recursos de maneira consciente e eficiente, a fim de evitar cobranças desnecessárias. Certifiquem-se de que as instâncias estejam ativas apenas quando necessárias e priorizem atividades acadêmicas relevantes. O prazo para o consumo desses créditos é determinado; portanto, fiquem atentos ao período disponível para garantir que possamos aproveitar ao máximo essa oportunidade.

A PUC-Rio não se responsabiliza por despesas advindas das atividades dos alunos na nuvem de sua escolha.